Actividad 03

Tema: Algoritmos en grafos

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Docente	Tema	Duración
Harvey A. Norman Patrick	Algoritmos en grafos	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	12/10/2026	–/10/2026

Índice

1. Actividades **3**
1.1. Comparación de algoritmos 3

2. Referencias bibliográficas **6**

1. Actividades

Para los algoritmos de Borůvka, Jarník, Kruskal, Dijkstra y Floyd realice una tabla comparativa de tiempos de ejecución para 10 nodos, analizando:

- Complejidad temporal
- Complejidad espacial
- Casos de uso, cuándo usar y cuándo no

1.1. Comparación de algoritmos

Tabla 1: Complejidad temporal y espacial para grafos con $n = 10$ nodos

Algoritmo	Tiempo de ejecución ($n = 10$)	Complejidad temporal	Complejidad espacial
Borůvka Borůvka (1926)	$\approx$ 149 operaciones (grafo denso, $E \log V$)	$O(E \log V)$	$O(V + E)$
Jarník (Prim) Jarník (1930)	100 operaciones con matriz; $\approx$ 149 con cola de prioridad	$O(V^2)$ con matriz; $O(E \log V)$ con cola de prioridad; $O(E + V \log V)$ con montículo de Fibonacci	$O(V^2)$ con matriz; $O(V + E)$ con lista de adyacencia
Kruskal Kruskal (1956)	$\approx$ 247 operaciones ($E \log E$)	$O(E \log E) = O(E \log V)$	$O(V + E)$ (lista de aristas + unión-búsqueda)
Dijkstra Dijkstra (1959)	100 operaciones con matriz; $\approx$ 183 con cola de prioridad	$O(V^2)$ con matriz; $O((V+E) \log V)$ con montículo binario; $O(E + V \log V)$ con montículo de Fibonacci	$O(V^2)$ con matriz; $O(V + E)$ con lista de adyacencia
Floyd (Floyd–Warshall) Floyd (1962)	1000 operaciones (V^3)	$O(V^3)$ (fija, sin variantes relevantes)	$O(V^2)$ (matriz de distancias)

Tabla 2: Casos de uso: cuándo usar y cuándo no usar cada algoritmo

Algoritmo	Cuándo usar	Cuándo no usar
Borůvka Borůvka (1926)	Construir un árbol de expansión mínima (MST) en grafos densos o en entornos paralelos/distribuidos, ya que procesa todos los componentes simultáneamente en cada ronda; es la base de algoritmos paralelos modernos de MST.	Si solo se necesita el MST de un grafo pequeño o disperso, Kruskal o Prim son más simples de implementar y depurar; tampoco aplica si el grafo tiene pesos negativos relevantes para otro objetivo (el MST no depende del signo, pero el algoritmo no resuelve caminos mínimos).
Jarník (Prim) Jarník (1930)	Construir el MST de grafos densos representados con matriz de adyacencia, o cuando conviene crecer el árbol desde un único vértice (por ejemplo, redes donde se parte de un nodo central).	En grafos dispersos representados con listas de aristas, Kruskal suele ser más eficiente y más simple; tampoco sirve si se requieren caminos mínimos (solo da un árbol de costo mínimo, no las distancias más cortas entre pares).
Kruskal Kruskal (1956)	Construir el MST de grafos dispersos, o cuando las aristas ya están ordenadas o es fácil ordenarlas; es sencillo de combinar con una estructura de conjuntos disjuntos (unión-búsqueda).	En grafos muy densos, el costo de ordenar $E \log E$ aristas puede superar al de Prim con matriz ($O(V^2)$); tampoco aplica si el grafo no es conexo y se requiere un único árbol (produce un bosque de expansión mínima en ese caso).
Dijkstra Dijkstra (1959)	Calcular el camino más corto desde un único origen hacia todos los demás nodos, en grafos con pesos no negativos; es la opción estándar para ruteo en redes y mapas.	Si el grafo tiene aristas de peso negativo, el algoritmo puede dar resultados incorrectos (no está diseñado para eso); tampoco conviene si se necesitan los caminos más cortos entre todos los pares de nodos y el grafo es denso, donde Floyd–Warshall resulta más directo.
Floyd (Floyd–Warshall) Floyd (1962)	Calcular los caminos más cortos entre todos los pares de nodos a la vez, incluso con pesos negativos (siempre que no haya ciclos negativos); útil en grafos pequeños o densos donde $O(V^3)$ es aceptable.	En grafos grandes o dispersos, el costo $O(V^3)$ resulta excesivo comparado con ejecutar Dijkstra desde cada nodo ($O(V \cdot (V + E) \log V)$); tampoco es la opción natural si solo se necesita el camino más corto desde un único origen.

2. Referencias bibliográficas

Borůvka, O. (1926). O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, *3*(3), 37–58.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*(1), 269–271. doi: 10.1007/BF01386390

Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, *5*(6), 345. doi: 10.1145/367766.368168

Jarník, V. (1930). O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, *6*, 57–63.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, *7*(1), 48–50. doi: 10.1090/S0002-9939-1956-0078686-7